

JNX Learning Platform - Abacus Agent Onboarding

Purpose: This document provides a structured guide for new Abacus AI chat sessions to understand and extend the JNX Learning Platform.

Last Updated: 2024-12-29

Phase: Building Qryx (First Product) - Phase 4 Complete | Phase 5 In Progress

Status: Backend Ready | **CRITICAL:** SaaS Flow Redesign Required

First Product: Qryx (Shopify AI Sales Assistant)

CRITICAL DISCOVERY (2024-12-29):

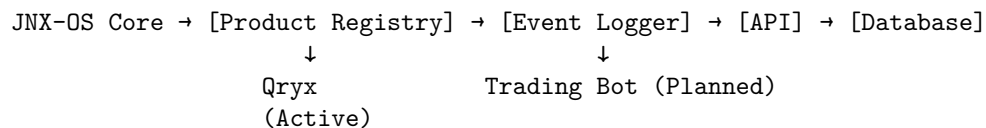
The original installation flow was **fundamentally incorrect**. This is NOT a simple Shopify OAuth app - it's a **full SaaS product with billing, subscriptions, and multi-product support**. See "Phase 5 Requirements" section below for complete details.

What is JNX Learning Platform?

TL;DR: A centralized AI learning system that collects data from all JNX products (Qryx, Trading Bot, etc.) to enable: - Pattern recognition - Performance optimization - Continuous improvement - Cross-product learning

Current Products: - **Qryx** (Phase 4 Complete) - Shopify AI Sales Assistant with Gemini 2.0 Flash - Database Schema: Deployed - API Endpoints: Implemented - Dashboard UI: Built - Shopify Integration: Awaiting configuration - Trading Bot (Planned) - More products coming...

Architecture:



Key Files: - lib/jnx-core/ - Core SDK (Registry, Logger, Types) - lib/jnx-products/ - Product configurations - app/api/jnx/events/route.ts - API endpoint - MIGRATION_JNX_LEARNING_PLATFORM.sql - Database schema

Task: Add New Product (3 Steps)

When to Use This:

- User says: "Integrate [Product Name] with JNX Learning Platform"
- User says: "Add [Product Name] to the learning system"

- User says: “Enable AI learning for [Product Name]”

Step 1: Create Product Configuration

Action: Create file `lib/jnx-products/[product-name]/config.ts`

Template:

```
import { z } from 'zod'
import { defineProduct } from '@lib/jnx-core/registry'

export default defineProduct({
  id: '[product_id]', // lowercase, underscores
  name: '[Product Display Name]', // human-readable
  version: '1.0.0',

  // Define events that this product will log
  events: {
    '[event_name]': {
      schema: z.object({
        // Define event structure
        field1: z.string(),
        field2: z.number().optional(),
      }),
      description: 'What this event tracks'
    }
  },

  // Paths AI must NEVER modify
  protected: [
    'core/*', // Core business logic
    'api/payment/*', // Payment processing
    'api/webhooks/*', // External integrations
  ],

  // Paths AI CAN optimize (with approval)
  optimizable: [
    'prompts/*', // AI prompts
    'ui/formatting', // UI improvements
    'performance/*', // Performance tuning
  ],

  // Optimization targets
  goals: {
    metricName: {
      target: 100, // Target value
      unit: 'ms' // Unit (ms, percentage, rating, etc.)
    }
  }
})
```

```

    }
  }
})

```

Example (Trading Bot):

```

import { z } from 'zod'
import { defineProduct } from '@lib/jnx-core/registry'

export default defineProduct({
  id: 'trading_bot',
  name: 'JNX Trading Bot',
  version: '1.0.0',

  events: {
    'trade_executed': {
      schema: z.object({
        symbol: z.string(),
        action: z.enum(['buy', 'sell']),
        amount: z.number().positive(),
        price: z.number().positive(),
        profit_loss: z.number(),
        strategy: z.string()
      }),
      description: 'Logged when a trade is executed'
    },
    'market_analysis': {
      schema: z.object({
        symbol: z.string(),
        indicators: z.record(z.number()),
        signal: z.enum(['bullish', 'bearish', 'neutral'])
      }),
      description: 'Market analysis results'
    }
  },

  protected: [
    'core/order-execution',
    'core/risk-management',
    'core/balance-check'
  ],

  optimizable: [
    'strategies/entry-timing',
    'strategies/exit-timing',
    'indicators/parameters'
  ],

```

```

goals: {
  profitability: { target: 0.15, unit: 'percentage' },
  drawdown: { target: 0.05, unit: 'percentage' },
  winRate: { target: 0.6, unit: 'percentage' }
}
})

```

Step 2: Register Product

Action: Edit lib/jnx-products/index.ts

Add this line:

```
import './[product-name]/config'
```

Full file example:

```

// Import all product configurations
import './qryx/config'
import './trading-bot/config'    // + Add this line

// Export registry utilities
export { registry, defineProduct, getProduct } from '@lib/jnx-core/registry'
export { useProductLogger, useLogEvent } from '@lib/jnx-core/hooks'

```

Step 3: Use in Code

Action: Add event logging to product code

Template:

```

import { useProductLogger } from '@lib/jnx-products'

function YourComponent() {
  const logger = useProductLogger('[product_id]')

  const handleEvent = async () => {
    await logger.logEvent('[event_name]', {
      // Event data matching your schema
      field1: 'value',
      field2: 123
    })
  }

  return <button onClick={handleEvent}>Action</button>
}

```

Example (Trading Bot):

```

import { useProductLogger } from '@lib/jnx-products'

function TradingDashboard() {
  const logger = useProductLogger('trading_bot')

  const executeTrade = async (trade: Trade) => {
    const result = await exchange.placeOrder(trade)

    // Log the trade
    await logger.logEvent('trade_executed', {
      symbol: trade.symbol,
      action: trade.action,
      amount: trade.amount,
      price: result.price,
      profit_loss: calculatePL(result),
      strategy: trade.strategy
    })
  }

  return <button onClick={() => executeTrade(...)}>Execute Trade</button>
}

```

Verification Checklist

After completing the 3 steps, verify:

```

# 1. TypeScript check
cd /home/ubuntu/jnx-os/nextjs_space
yarn tsc --noEmit
# Expected: No errors

# 2. Build check
yarn build
# Expected: Successful build

# 3. Verify product registered (in code)
import { getAllProducts } from '@lib/jnx-products'
console.log(getAllProducts())
// Expected: Your product appears in list

```

In database (Supabase SQL Editor):

```

-- Check if events are being logged
SELECT
  product_type,
  event_type,

```

```

COUNT(*) as count
FROM product_events
GROUP BY product_type, event_type
ORDER BY product_type;

-- Expected: Your product_type appears with event counts

```

Best Practices

Event Schema Design

Good:

```

events: {
  'user_action': {
    schema: z.object({
      action: z.enum(['click', 'submit', 'cancel']),
      timestamp: z.string(),
      duration_ms: z.number().positive().optional()
    })
  }
}

```

Bad:

```

events: {
  'event1': { // Not descriptive
    schema: z.object({
      data: z.any(), // Too generic
      stuff: z.string() // Unclear meaning
    })
  }
}

```

Protected vs Optimizable Paths

Protected (AI can NEVER modify): - Core business logic - Payment processing - Authentication - Database operations - Security features - External API integrations

Optimizable (AI CAN suggest changes): - AI prompts and templates - UI formatting and styling - Performance optimizations - Caching strategies - Response formatting - User experience improvements

Goal Setting

Good Goals (SMART):

```
goals: {
  responseTime: { target: 2000, unit: 'ms' },           // Specific, Measurable
  accuracy: { target: 0.95, unit: 'percentage' },       // Clear target
  userSatisfaction: { target: 4.5, unit: 'rating' }      // Achievable
}
```

Bad Goals:

```
goals: {
  fast: { target: 1, unit: 'yes' },                     // Not measurable
  good: { target: 100, unit: 'good' },                  // Unclear unit
  best: { target: 999, unit: 'best' }                   // Not achievable
}
```

Troubleshooting

Error: “Product ‘xyz’ not found”

Cause: Product not registered in lib/jnx-products/index.ts

Fix:

```
// lib/jnx-products/index.ts
import './xyz/config' // Add this line
```

Error: “Event type ‘abc’ not defined”

Cause: Event not in product config

Fix:

```
// lib/jnx-products/your-product/config.ts
events: {
  'abc': { // Add this event
    schema: z.object({ ... })
  }
}
```

Error: Validation Failed

Cause: Event data doesn’t match schema

Fix: Check Zod error message for details:

```
try {
  await logger.logEvent('event_name', data)
} catch (error) {
  console.error('Validation error:', error)
```

```
// Error will show which field failed and why  
}
```

Events Not Appearing in Database

Check: 1. Migration executed? Run `MIGRATION_JNX_LEARNING_PLATFORM.sql` in Supabase 2. User authenticated? Events require Clerk authentication 3. API endpoint working? Check `/api/jnx/events` logs

Reference Files

Core SDK

- `lib/jnx-core/types.ts` - All TypeScript types
- `lib/jnx-core/registry.ts` - Product registry system
- `lib/jnx-core/event-logger.ts` - Event logging with PII redaction
- `lib/jnx-core/hooks.ts` - React hooks for easy use

Example Product

- `lib/jnx-products/qryx/config.ts` - Complete Qryx configuration
- `lib/jnx-products/README.md` - Detailed guide with examples

API & Database

- `app/api/jnx/events/route.ts` - API endpoint
- `MIGRATION_JNX_LEARNING_PLATFORM.sql` - Database schema

Documentation

- `JNX_LEARNING_PLATFORM_PHASE1.md` - Complete Phase 1 guide
 - `lib/jnx-products/README.md` - Product integration guide
-

Workflow for New Chat Session

If user requests new product integration:

1. **Understand the product:**
 - What does it do?
 - What events should be logged?
 - What are optimization goals?
 - What paths are protected?
2. **Follow 3-step process:**
 - Create config file
 - Register in `index.ts`

- Add logging to code
 - 3. **Verify:**
 - TypeScript check
 - Build check
 - Database check
 - 4. **Document:**
 - Update this file if patterns change
 - Add to `lib/jnx-products/README.md` if needed
 - 5. **Save checkpoint:**
 - Build successful
 - All tests passing
 - Documentation updated
-

Important Notes

DO:

- Follow the 3-step process exactly
- Use Zod for schema validation
- Keep event names descriptive
- Set realistic, measurable goals
- Test TypeScript before committing
- Verify events in database

DON'T:

- Modify core SDK files (`lib/jnx-core/*`)
 - Change API endpoint (`app/api/jnx/events/route.ts`)
 - Use **any** types in schemas
 - Skip validation
 - Forget to register in `index.ts`
 - Log PII without redaction (automatic in SDK)
-

Success Criteria

Product integration is complete when:

1. Config file created in `lib/jnx-products/[product-name]/config.ts`
2. Product registered in `lib/jnx-products/index.ts`
3. Event logging added to product code
4. TypeScript: No errors
5. Build: Passing
6. Events visible in database
7. Checkpoint saved

Support

For new chat sessions: - Read this file first - Check `lib/jnx-products/README.md` for examples - Review `JNX_LEARNING_PLATFORM_PHASE1.md` for architecture - Look at `lib/jnx-products/qryx/config.ts` for reference

Common Questions: - “How do I add a new product?” → Follow 3-step process above - “What events should I log?” → Depends on product, see examples - “What are protected paths?” → Core logic, payment, auth, security - “How do I set goals?” → Use SMART criteria (Specific, Measurable, etc.)

Version: 1.0.0

Phase: 1 Complete

Status: Production-Ready

Last Updated: 2024-12-28

Phase 5: SaaS Installation Flow & Billing (IN PROGRESS)

Critical Redesign Required

Discovery Date: 2024-12-29

Issue: The original Qryx installation flow was designed as a **direct Shopify OAuth app**, but the business model requires a **full SaaS platform with billing**.

What Was Wrong:

Shopify App Store → Install → OAuth → Done

This would allow anyone to install without payment or account creation.

Correct Multi-Product SaaS Flow:

1. Shopify App Store → "Install Qryx"
↓
2. `www.jnxlabs.ai/api/qryx/install?shop=merchant.myshopify.com`
↓
3. Save shop parameter in session (encrypted cookie/DB)
↓
4. Redirect to `/login` (or `/signup` for new users)
↓
5. NEW USER PATH:
 - Registration form (email, password, name)

- Clerk User created
- Clerk Organization created
- ↓
- 6. Redirect to /products/qryx/setup
- ↓
- 7. PRODUCT SELECTION & PRICING:
- QRYX - AI Sales Assistant
- STARTER - \$29/mo
- PROFESSIONAL - \$99/mo
- ENTERPRISE - Custom
- ↓
- 8. User selects plan → "Subscribe"
- ↓
- 9. STRIPE CHECKOUT SESSION
 - Customer payment info
 - Subscription created
 - Webhook: checkout.session.completed
- ↓
- 10. Payment Success → Subscription in DB
- ↓
- 11. NOW: Trigger Shopify OAuth (retrieve saved shop parameter)
- ↓
- 12. Merchant approves OAuth permissions
- ↓
- 13. CALLBACK: /api/qryx/callback
 - Link shop to user's organization
 - Install widget (Script Tag)
 - Create initial configuration
- ↓
- 14. Redirect to Dashboard
 - Qryx Connected
 - Subscription status visible
 - Widget configuration available
 - Analytics enabled

Required Components (Not Yet Built):

1. Shop Session Management **File:** lib/session/shop-session.ts
Purpose: Preserve the shop parameter through multiple redirects
Implementation: - Encrypted session cookie OR - Database state table

2. Product Selection Page **File:** app/products/qryx/setup/page.tsx
Features: - Display Qryx pricing tiers - Plan comparison table - "Subscribe"

button per plan - Check if user already has subscription

3. Stripe Integration Files Required: - lib/stripe/client.ts - Stripe SDK setup - app/api/stripe/checkout/route.ts - Create checkout session - app/api/stripe/webhook/route.ts - Handle subscription events - lib/db/billing-helpers.ts - Subscription CRUD operations

Stripe Webhooks to Handle: - checkout.session.completed → Create subscription in DB - customer.subscription.updated → Update plan/status - customer.subscription.deleted → Cancel subscription - invoice.payment_succeeded → Extend subscription - invoice.payment_failed → Suspend access

4. Installation Flow Update CRITICAL File: app/api/qryx/install/route.ts
Current Behavior (WRONG):

```
// Directly redirects to Shopify OAuth
const authUrl = await getAuthorizationUrl(shop, state);
return NextResponse.redirect(authUrl);
```

Required Behavior:

```
// Save shop parameter
await setShopSession(shop);

// Redirect to login
const loginUrl = new URL('/login', request.url);
loginUrl.searchParams.set('redirect', '/products/qryx/setup');
return NextResponse.redirect(loginUrl);
```

5. OAuth Trigger After Payment File: app/api/qryx/start-oauth/route.ts
(NEW)

Purpose: Initiated AFTER successful Stripe payment

Logic: 1. Verify user has active subscription 2. Retrieve saved shop parameter
3. Generate OAuth URL 4. Return to frontend for redirect

6. Subscription Management UI File: app/app/billing/page.tsx

Features: - Current plan display - Usage statistics (conversations used / limit)
- Billing history - Upgrade/Downgrade options - Cancellation flow - Invoice download

Implementation Priority:

Phase 5A - Core Flow (NEXT): 1. Shop session management (lib/session/shop-session.ts) 2. Update installation flow (app/api/qryx/install/route.ts)
3. Product selection page (app/products/qryx/setup/page.tsx)

4. Stripe checkout integration 5. OAuth trigger after payment
(`app/api/qryx/start-oauth/route.ts`)

Phase 5B - Subscription Management: 1. Stripe webhook handler 2. Billing dashboard UI 3. Usage tracking 4. Plan upgrade/downgrade logic

Phase 5C - Polish: 1. Error handling & recovery 2. Email notifications (payment success/failed) 3. Admin subscription management 4. Analytics & reporting

Test Environment:

- **Test Shop:** `shopbotv3.myshopify.com`
- **Production URL:** `https://www.jnslabs.ai`
- **Shopify App URL:** Should be set to `https://www.jnslabs.ai/api/qryx/install`
- **Redirect URI:** `https://www.jnslabs.ai/api/qryx/callback`
(already configured)

Current Blockers:

- No Stripe integration (SDK, checkout, webhooks)
- No product selection/pricing page
- Installation flow bypasses login/payment
- No subscription management system
- No billing dashboard

Success Criteria for Phase 5:

- ☐ User can install Qryx from Shopify App Store
- ☐ User is prompted to login/register at JNX
- ☐ User can view and select pricing plans
- ☐ Payment is processed securely via Stripe
- ☐ Subscription is created in database
- ☐ OAuth happens ONLY after successful payment
- ☐ Widget is installed automatically
- ☐ Dashboard displays subscription status
- ☐ User can manage billing settings

Related Documentation:

- See `CONVERSATION_STARTER.md` for detailed flow diagrams
- See `ENVIRONMENT_VARIABLES_STATUS.md` for Shopify config status
- See `docs/QRYX_PRICING_STRATEGY.md` for pricing tiers (if exists)

Version: 1.0.0

Phase: 4 Complete | 5 In Progress

Status: Backend Ready | SaaS Flow Redesign Required
Last Updated: 2024-12-29